

IIT Hyderabad
Introduction to Programming
End sem examination – 2024
(Set A)

Total Marks: 73

Time duration: 3 hours

Full Roll No.: _____

Instructions

1. Please fill in your roll number correctly and legibly in the space above. Write your full roll number, including department code, etc.
2. Do NOT turn over this page till the start of the exam is announced (6 pm). Once the exam starts, you may turn over and read the questions overleaf.
3. Please ensure you are carrying only a pen/pencil and your ID card with you.
4. Do NOT carry any electronic devices like mobiles, watches, etc. In case invigilators find any device with you or any loose sheets/notes, you will be awarded FR, even if you did not intend to use it.
5. Please write your answers in this question paper itself. Please write legibly and within the space provided. Submit the filled-in question paper at the end of the exam to the invigilator (9 pm).
6. In the unlikely situation where you don't have space in the question paper, you may write on a blank sheet and submit it. Do not forget to write your roll number on such a page. Staple it along with this question paper.
7. You may use blank sheets given to you for rough work, which should also be submitted back to us. Staple it along with this question paper. Do not forget to write your roll number on such a page. Unidentified rough sheets found near you may make you a suspect for copying! Please be careful.
8. Any attempt to malpractice (copy, cheat, etc.) will straight away lead to an FR grade in the course.
9. Total marks is 73, but it will be normalized to 25% as announced.
10. After finishing your exam at 9 pm, stop writing. Do not write anything beyond 9 pm. Leave your filled question paper as it is on your desk and move away immediately towards the big screen. The invigilators will collect your sheets and do a tally. After they announce, you can leave the exam hall.

1 Multiple Choice Questions

Q.1) What will be the output of the following code snippet?

Solution: (C)

2 marks

A. 1 B. 2 C. 3 D. Undefined Behaviour E. None of the Above

```
1 #include <stdio.h>
2
3 struct Node {
4     int data;
5     struct Node* next;
6 };
7
8 int main() {
9     struct Node n1 = {1, NULL}, n2 = {2, NULL}, n3 = {3, NULL};
10    n1.next = &n2; n2.next = &n3; n3.next = &n1;
11    struct Node* temp = &n1;
12    for (int i = 0; i < 5; i++) temp = temp->next;
13    printf("%d", temp->data);
14    return 0;
15 }
```

Q.2) What will be the output of the following code snippet?

Solution: (B)

2 marks

A. 28 27 B. 10 27 C. 10 7 D. 64 27 E. None of the Above

```
1 #include <stdio.h>
2 #define SQUARE(x) x * x
3 #define CUBE(x) SQUARE(x) * (x)
4
5 int main() {
6     int result1 = CUBE(3 + 1);
7     int a = 2 + 1;
8     int result2 = CUBE(a);
9     printf("%d %d", result1, result2);
10    return 0;
11 }
```

Q.3) What will be the output of the following code snippet?

Solution: (A)

2 marks

A. o world B. l world C. o hello D. Compilation error E. None of the Above

```
1 #include <stdio.h>
2
3 int main() {
4     char *arr[] = {"hello", "world"};
5     char **ptr = arr;
6     printf("%c %s", *(*ptr + 4), *(ptr + 1));
7     return 0;
8 }
```

Q.4) What will be the output of the following code snippet?

Solution: (A)

2 marks

A. 40 30 20 50 B. 30 20 40 10 C. 40 20 30 50 D. Undefined Behaviour E. None of the Above

```
1 #include <stdio.h>
2
3 int main() {
4     int arr[5] = {10, 20, 30, 40, 50};
5     int *p = arr;
6
7     int x = arr[3];
8     int y = *(arr + 2);
9     int z = p[1];
10    int w = *(p + 4);
11
12    printf("%d %d %d %d", x, y, z, w);
13    return 0;
14 }
```

Q.5) What will be the output of the following code snippet?

Solution: (C)

2 marks

A. 1 B. 6 C. 4 D. 9 E. None of the Above

```
1 #include <stdio.h>
2
3 int main() {
4     unsigned int x = 9; //binary of 9 is 000...001001
5     unsigned int y = 12; //binary of 12 is 000...001100
6     printf("%u", (~x | y) & (x ^ y));
7     return 0;
8 }
```

Q.6) What will be the output of the following code snippet?

Solution: (C)

2 marks

A. HelloGo B. HelGo C. HelloHelloGo D. Undefined Behaviour E. None of the Above

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char str1[] = "Hello";
6     char str2[] = "Hello";
7     char str3[20] = "";
8     int result = strcmp(str1, str2);
9     if(result)
10    {
11        strncpy(str3, str1, 3);
12    }
13    else
14    {
15        strcat(str3, str1);
16        strcat(str3, str2);
17    }
18 }
```

```

17     }
18     printf("%s", strcat(str3, "Go"));
19     return 0;
20 }

```

Q.7) What will be the output of the following code snippet?

Solution: (B)

2 marks

A. 2 B. 3 C. 4 D. 5 E. None of the Above

```

1 #include <stdio.h>
2
3 int main(){
4     int arr[] = {1, 2, 3, 4, 5};
5     int sum = 0;
6     for (int i = 0; i < 5; i++) {
7         if (i % 2 == 0) {
8             sum += arr[i];
9         } else {
10            sum -= arr[i];
11        }
12    }
13    printf("%d", sum);
14    return 0;
15 }

```

Q.8) What will be the output of the following code snippet?

Solution: (B)

2 marks

A. 0 2 4 8 B. 2 4 6 8 C. 2 2 3 4 D. Undefined Behaviour E. None of the Above

```

1 #include <stdio.h>
2
3 void func(int *arr, int n) {
4     if (n <= 0) return;
5     *(arr + n - 1) = *(arr + n - 1) * 2;
6     func(arr, n - 1);
7 }
8
9 int main() {
10    int arr[] = {1, 2, 3, 4};
11    func(arr, 4);
12    printf("%d %d %d %d", arr[0], arr[1], arr[2], arr[3]);
13    return 0;
14 }

```

Q.9) What will be the output of the following code snippet?

Solution: (B)

2 marks

A. 1 2 3 B. 3 2 1 C. 1 1 1 D. 2 1 3 E. None of the Above

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;

```

```

6     struct Node *next;
7 };
8
9 void foo(struct Node *head) {
10     if (head == NULL) return;
11     foo(head->next);
12     printf("%d ", head->data);
13 }
14
15 int main() {
16     struct Node *head = (struct Node*)malloc(sizeof(struct Node));
17     head->data = 1;
18     head->next = (struct Node*)malloc(sizeof(struct Node));
19     head->next->data = 2;
20     head->next->next = (struct Node*)malloc(sizeof(struct Node));
21     head->next->next->data = 3;
22     head->next->next->next = NULL;
23
24     foo(head);
25     return 0;
26 }

```

Q.10) What will be the output of the following code snippet?

Solution: (B)

2 marks

A. 15 B. 18 C. 21 D. 23 E. None of the Above

```

1 #include <stdio.h>
2
3 int foo(int n) {
4     if (n <= 1) return 1;
5     return n + foo(n / 2);
6 }
7
8 int main() {
9     printf("%d", foo(10));
10    return 0;
11 }

```

2 Numerical Questions: Output Prediction

Q.1) Consider the following code snippet:

2 marks

```

1 int A[2][2] = {{7, 2}, {4, 4}};
2 int B[2][2] = {{5, 6}, {4, 8}};
3 int C[2][2];
4 for (int i = 0; i < 2; i++) {
5     for (int j = 0; j < 2; j++) {
6         C[i][j] = A[i][0] * B[0][j] + A[i][1] * B[1][j];
7     }
8 }

```

What will be the resulting matrix C?

Solution:

43 58

36 56

Q.2) Consider the following code snippet:

2 marks

```
1 int doSomething(int n, int x) {  
2     if (n == 0) return x;  
3     return doSomething(n / 10, x * 10 + (n % 10));  
4 }
```

Predict the output of doSomething(1234, 0):

Solution: 4321

Q.3) Consider the following code snippet:

2 marks

```
1 #include <stdio.h>  
2  
3 typedef struct Pair{  
4     int x;  
5     double y;  
6 } Pair;  
7  
8 int main(){  
9     printf("%lu\n", sizeof(Pair*)/sizeof(int*));  
10 }
```

What will be the output of this code?

Solution: 1

Q.4) Consider the following code snippet:

2 marks

```
1 #include <stdio.h>  
2 #include <string.h>  
3 int main() {  
4     char str[] = "12345";  
5     int num = 0;  
6  
7     for (int i = 0; i < strlen(str); i++) {  
8         num = num * 10 + (str[i] - '0');  
9     }  
10  
11     printf("%d %c", num, str[num % 5]);  
12     return 0;  
13 }
```

What will be the output of this code?

Solution: 12345 1

Q.5) Consider the following code snippet:

3 marks

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3  
4 int main() {  
5     int *arr = (int *)malloc(4 * sizeof(int));
```

```

6   for (int i = 0; i < 4; i++) {
7       *(arr + i) = (i + 1) * 5;
8   }
9
10  int *p = arr, *q = arr + 3;
11  int result = 0;
12
13  while (p <= q) {
14      result += (*p) - (*q);
15      p++;
16      q--;
17  }
18
19  free(arr);
20  printf("%d \n", result);
21  printf("%d", *q);
22  return 0;
23 }

```

What will be the output of this code?

Solution: -20 and second printf may print garbage value or may lead to runtime error

3 Write Short Answer / Fill in the blanks

Q.1) Consider the following function definition.

1 mark

```

1 void doSomething(int arr[], int n) {
2     int start = 0, end = n-1;
3     while (start < end) {
4         int temp = arr[start];
5         arr[start] = arr[end];
6         arr[end] = temp;
7         start++;
8         end--;
9     }
10 }

```

If the input array is **arr[] = {1, 2, 3, 4, 5, 6, 7}**, then what will be the elements in the modified array, **arr[]**, after the function doSomething is called?

Solution: {7, 6, 5, 4, 3, 2, 1 }

Q.2) Consider the below code snippet:

2 marks

```

1 int find(int arr[], int n, int k) {
2     int count[100001] = {0};
3     int left = 0, maxLen = 0, distinct = 0;
4
5     for (int right = 0; right < n; ++right) {
6         if (count[arr[right]] == 0) {
7             distinct++;
8         }
9         count[arr[right]]++;
10    }

```

```

11     while (distinct > k) {
12         count[arr[left]]--;
13         if (count[arr[left]] == 0) {
14             distinct--;
15         }
16         left++;
17     }
18     int currentLen = right - left + 1;
19     if (currentLen > maxLen) {
20         maxLen = currentLen;
21     }
22 }
23 return maxLen;
24 }

```

If the input array is `int arr[] = {1, 2, 1, 2, 3, 4, 5, 2, 2, 3};`, the value of `n = 10`, and `k = 2`, what value will the function `find` return? Justify your answer in one sentence.

Solution: return value is 4, corresponding to {1, 2, 1, 2}

Explanation: This function is finding the length of the longest subarray having atmost k distinct elements.

Q.3) Analyse the `do_something` function defined below to answer the following question.

3 marks

```

1 #include <stdlib.h>
2 #include <stdio.h>
3
4 struct ListNode{
5     double data;
6     struct ListNode *next;
7 };
8
9 struct ListNode* do_something(struct ListNode* head, int k) {
10     struct ListNode *dummy = (struct ListNode *)malloc(sizeof(struct ListNode));
11     dummy->next = head;
12     struct ListNode *groupPrev = dummy, *groupEnd = dummy;
13     while (groupEnd->next != NULL) {
14         // Move groupEnd to the k-th node
15         for (int i = 0; i < k && groupEnd != NULL; ++i) {
16             groupEnd = groupEnd->next;
17         }
18         if (groupEnd == NULL) break;
19         struct ListNode* groupStart = groupPrev->next;
20         struct ListNode* nextGroup = groupEnd->next;
21         groupEnd->next = NULL;
22         struct ListNode* prev = NULL;
23         struct ListNode* curr = groupStart;
24         while (curr != NULL) {
25             struct ListNode* next = curr->next;
26             curr->next = prev;
27             prev = curr;
28             curr = next;
29         }
30         groupPrev->next = prev; // prev is the new head of the reversed group
31         groupStart->next = nextGroup;

```



```

32     groupPrev = groupStart;
33     groupEnd = groupPrev;
34 }
35 return dummy->next;
36 }

```

If the input linked list to the function `do_something` is: **head = 1 -> 2 -> 3 -> 4 -> 5 -> 6->7->8 -> NULL**, and **k = 3**, what will the modified linked list be after the function call? In other words, if we print the data stored in the linked list, whose root node's pointer is returned by `do_something`, then what will be printed?

Solution: 3 -> 2 -> 1 -> 6 -> 5 -> 4 -> 7 -> 8 -> NULL

Q.4) Fill in the blanks (A), (B) with statements involving a conditional/ternary operator (?:) in the following C code snippet, so that the returned value will correctly find the product of numbers in the **contiguous subarray with the largest product**. Assume that all the numbers in the array are positive. Do not implement the function using nested for loops. Hint: think, when do you need to reset cumulativeProduct to the current number in the array instead of carrying over the previously cumulated product.

4 marks

```

1 double maxProductSubarray(double arr[], int n) {
2     double maximumProduct = arr[0];
3     double cumulativeProduct = arr[0];
4     for (int i = 1; i < n; ++i) {
5         ----- (A) -----
6         ----- (B) -----
7     }
8     return maximumProduct;
9 }

```

Blanks to fill:

(A): **cumulativeProduct = (cumulativeProduct < 1) ? arr[i] : cumulativeProduct * arr[i];**

(B): **maximumProduct = (maximumProduct < cumulativeProduct) ? cumulativeProduct : maximumProduct;**

Note: Anything equivalent is also correct.

4 Logical and Runtime Errors

Note: While answering questions in this section, assume that all the necessary header files are included and there are no issues with them. Also, assume that inputs from the user (if any) are given correctly. Your changes must ensure that the code works as intended on all machines/compilers and for all valid inputs.

Q.1) The following program was written to check whether a given word is a palindrome or not. A word is called a palindrome if it appears the same whether read forwards or backwards. For example, the word "madam" (without quotes) or the exclamation "heh" (without quotes). In comparison, the word "sir" (without quotes) is not a palindrome. The following program was written to check if a given word is a palindrome, but it has errors. Identify the bug/issue in the code and write the fixes in the given box (no need to rewrite the entire code).

2 marks

```

1 int main () {
2     char str[100];
3     scanf("%s", str);
4     int p, sta = 0, fin;
5     for(p=0;p<100;p++){
6         if(str[p] == '\n')
7             break;
8     }
9     fin = p;
10    int isPalindrom = 0;
11    while(sta < fin){
12        if(str[sta] != str[fin]){
13            isPalindrom = 1;
14            break;
15        }
16        sta++;
17        fin--;
18    }
19    if(isPalindrom)
20        printf("YES");
21    else
22        printf("NO");
23    return 0;
24 }

```

Solution: :

- (a) Line 6: '\0' instead of '\n'
- (b) Line 9: p-1
- (c) Line 10: isPalindrom = 1;
- (d) Line 13: isPalindrom = 0;

Q.2) The program is intended to reverse a given string but produces incorrect or unexpected results. Review the code, identify the underlying issue, and make the necessary corrections to ensure it functions as expected. Write the fixes in the given box (no need to rewrite the entire code).

2 marks

```

1 int main() {
2     char str[100];
3     scanf("%s", str);
4     int len = strlen(str);
5     char reversed[len];
6     for (int i = 0; i < len; i++) {
7         reversed[i] = str[len - i - 1];
8     }
9     printf("Reversed string: %s\n",
10    reversed);
11    return 0;
12 }

```

Solution: :

- (a) Line 5: char reversed[len+1];
- (b) Line 8: } reversed[len]='\0';

Q.3) The given function is intended to rotate an array to the right by k (≥ 0) positions e.g., if input arr is d_1, d_2, d_3, d_4, d_5 and $k = 3$, after executing this function, arr must contain d_3, d_4, d_5, d_1, d_2 . However, for certain values of k , the output is incorrect. Please identify the bug, and update the code to ensure the array is correctly rotated for all valid inputs. Write the fixes in the given box (no need to rewrite the entire code).

2 marks

```

1 void rotateArray(int arr[], int n, int k)
  {
2     k = k % n;
3     int temp[n];
4     for (int i = 0; i < n; i++) {
5         temp[i + k] = arr[i];
6     }
7     for (int i = 0; i < n; i++) {
8         arr[i] = temp[i];
9     }
10 }

```

Solution: :

(a) Line 5: temp[(i + k)%n] = arr[i];

Q.4) The function is intended to merge two sorted linked lists into a single sorted linked list. However, when provided with inputs l1: 1 -> 5 -> NULL and l2: NULL, it fails to return the correct result. Please identify the issue in the code and correct it. Write the fixes in the given box (no need to rewrite the entire code).

2 marks

```

1 struct Node {
2     int data;
3     struct Node* next;
4 };
5
6 struct Node* mergeLists(struct Node* l1,
7     struct Node* l2) {
8     struct Node* merged = NULL;
9     if (l1->data < l2->data) {
10         merged = l1;
11         l1 = l1->next;
12     } else {
13         merged = l2;
14         l2 = l2->next;
15     }
16     struct Node* current = merged;
17     while (l1 != NULL && l2 != NULL) {
18         if (l1->data < l2->data) {
19             current->next = l1;
20             l1 = l1->next;
21         } else {
22             current->next = l2;
23             l2 = l2->next;
24         }
25         current = current->next;
26     }
27     if (l1 != NULL) {
28         current->next = l1;
29     }
30     if (l2 != NULL) {
31         current->next = l2;
32     }
33     return merged;
34 }

```

Solution: :

Enclose block line 8-14 (say it block B), After that correction is as follows after the line 7 and before line 15:

```

1 if(l1==NULL && l2==NULL) {
2     return NULL;
3 }
4 else if(l1==NULL) {
5     return l2;
6 }
7 }
8 else if(l2==NULL) {
9     return = l1;
10 }
11 else {
12     block B Code
13 }

```

Q.5) The given program is designed to identify and print all the distinct elements in an array. Note that the program must work for any array arr, not only for the one in the code. However, an

array containing duplicate entries, only prints nonrepeating elements, leading to incorrect output. For example, if the input array is **[1, 2, 3, 4, 2, 1, 5]**, then it prints **[3, 4, 5]** only, whereas the output should be **[1, 2, 3, 4, 5]**, the order in which they appear in the array. Analyze the code, identify the logical issue, and provide a corrected implementation. Write the fixes in the given box (no need to rewrite the entire code).

2 marks

```
1 int main() {
2     int arr[] = {1, 2, 3, 4, 2, 1, 5};
3     int n = 7;
4     for (int i = 0; i < n; i++) {
5         int isUnique = 1;
6         for (int j = 0; j < n; j++) {
7             if (arr[i] == arr[j] && i !=
8 j) {
9                 isUnique = 0;
10                break;
11            }
12            if (isUnique) {
13                printf("%d ", arr[i]);
14            }
15        }
16        return 0;
17 }
```

Solution: :

(a) Line 6: for (int j = 0; j < i; j++)

or

Line 6: for (int j = 0; j <= i; j++)

5 Multiple Select Questions

Note: Tick/Mark all the correct options in each question.

Q.1) Which of the following statements are **false**?

3 marks

Solution: B, C, D

A. Assume a machine stores `int` and `float` using 4 bytes. Let, `i`, be an `int` pointer. Let `j=(float *) i`. Then, the address pointed by `i + 1` is same as that pointed by `j + 1`.

B. Let, `f`, be an `float` pointer. Let `d=(double *) f`. Then, the address pointed by `f + 1` is same as that pointed by `d + 1`.

C. Assigning an `double` pointer the memory block allocated by `malloc(8 * sizeof(int*))` and typecasted by `double*` will render a compilation error/warning since `int` and `double` are two different types that consume different amounts of memory.

D. The code given below prints "Good!", without any compilation or run-time error.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     char* str;
7     str = (char*)malloc(6*sizeof(char));
8     str = "Hood!";
```

```

9     char ch = 'G';
10    str[0] = ch;
11
12    printf("%s\n", str);
13    return 0;
14 }

```

Q.2) Which of the following statements are **false** about a function pointer?

2 marks

Solution: B, D

A. A function pointer points to the start of the respective function's binary code i.e., it's value is the address of the starting instruction in the respective function's binary code.

B. If a pointer, p_{data} to a data variable is declared inside a function, then, outside the function one can refer to the pointer as $p_{func} \rightarrow p_{data}$, where p_{func} is a function pointer pointing to the function.

C. A function cannot be assigned to another function, but a function pointer can be assigned to another function pointer in C.

D. Functions cannot be passed as arguments to another function and function pointers also can't be passed as arguments either in C.

E. By default, a function pointer, when passed as an argument to a function in C, it's value (i.e, an address) is passed.

Q.3) Which of the following statements about the two codes given below (ref. A and B respectively) are **true**?

3 marks

Solution: A, B, C

Code A

```

1  #include <stdio.h>
2  #include <math.h>
3
4  void display(int *arr, int n) {
5      for(int index=0; index < n; index++)
6      {
7          if(arr[index]) {
8              printf("%d ", arr[index]);
9          }
10     }
11     printf("\n");
12 }
13
14 int main() {
15     int arr[] = {1, 2, 3, 4, 5, 6, 7, 8};
16     for(int i = 1; i <= 4; i++){
17         for(int j=1; j<= 8/i; j++) {
18             arr[j-1] += (i+j);
19         }
20     }
21     display(arr, 8);
22     return 0;
23 }

```

Code B

```

1  #include <stdio.h>
2  #include <math.h>
3
4  void display(int arr[], int n) {
5      for(int index=0; index < n; index++)
6      {
7          if(arr[index]) {
8              printf("%d ", arr[index]);
9          }
10     }
11     printf("\n");
12 }
13
14 int main() {
15     int arr[] = {1, 2, 3, 4, 5, 6, 7, 8};
16     for(int i = 1; i <= 8; i++){
17         for(int j=i; j<= 8; j*=2) {
18             arr[j-1] += (i+j);
19         }
20     }
21     arr[0] += 1;
22     display(arr, 8);
23     return 0;
24 }

```

A. The total number of arithmetic operations in the codes are not the same.

- B. The number of updates to elements in `arr` performed between lines 16 – 20 in code A are the same as those performed between lines 16 – 21 in code B.
- C. The output displayed by both programs is not the same.
- D. In C programming, whether we pass an array or a pointer to the array, in both cases all the elements of the array need to be copied within the function stack.

Q.4) Which of the following statements are **false**?

3 marks

Solution: B, D, E

A. Returning the address of a local data variable declared inside a function called inside `main()` and assigning it to a pointer declared in `main()`, even if it is of the same data type as the local variable, will lead to segmentation faults whenever the value of that pointer is referenced after the assignment.

B. When incrementing a pointer of type `T` `ptr` as `ptr++`, the pointer points to a location `sizeof(T*)` bytes away from the previous location. (Assume that the pointer only points to valid memory locations).

C.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char* str = (char*)malloc(5*sizeof(char));
6     scanf("%s", &str);
7     printf("%c", str[8]);
8     return 0;
9 }
```

Assume that the code above executes without segmentation fault. This is then dangerous since, it can potentially reveal some data written to by another user using the same memory resource.

D.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char* str = (char*)malloc(6*sizeof(char));
6     scanf("%s", &str);
7     printf("%c", str[8]);
8     return 0;
9 }
```

Assume that the code above executes without segmentation fault. There is no potential security breach, since the moment a pointer tries to access a memory allocation outside of what it was allocated, compiler assigns a garbage value.

E. The `main()` function cannot take any arguments when it is executing.

Q.5) Assume that a macro named "CHECK" is not defined and "NULL" is defined in the header file, "stdio.h". Which of the following statements are **false** for the given code snippet?

2 marks

Solution: A, C

```

1 #include <stdio.h>
2
3 int main() {
4
5     #ifndef CHECK
6     int x, y;
7     x = 0, y = 0;
8     #endif
9
10    printf("%d %d\n", x, y);
11    return 0;
12 }

```

- A. The above code will give a compilation error.
- B. If the line "#define CHECK 1" is add at line 2 in the above code, the code will give a compilation error.
- C. If lines 5 and 6 are swapped in the above code, garbage values will be printed for x and y.
- D. If "CHECK" were to be replaced with "NULL", and instead of "stdio.h", another library that calls "stdio.h" were to be included, it would give a compilation error.
- E. None of the above.

6 Paragraph Based Questions

6.1 Compiler Optimization

A compiler is a program that translates high-level code written by programmers into machine code that a computer can execute. During this translation, compilers also improve the efficiency of the code through a process called **optimization**. Compiler optimizations make programs faster, use less memory, or even reduce power consumption, without changing what the program does. For example, dead-code elimination is one kind of compiler optimization. In this optimization, parts of the code that will never execute, such as a function that is never called or a condition that is always false, are removed at compile time so that the code-size is smaller (requires less memory at execution time). Analogously, assignments which do not change an assigned variable are also discarded. Constant propagation is another optimization where the compiler replaces expressions involving variables known at compile time or constants with their values, so that they need not be computed at run-time. The exact optimizations that can be executed on a code vary between compilers. In this paragraph, however, we will restrict ourselves to the two optimizations discussed above: dead-code elimination and constant propagation.

Q.1) In the following code, which line numbers are likely to NOT be executed at run-time, after compiler optimization?

Solution: C, D, E

```

1 #include <stdio.h>
2
3 const int N = 6;
4 const int K = 9;
5 const int T = 9*N - 6*K;

```

```

6
7 int main() {
8
9     int array[5] = {5, 29, 4, 0, -1};
10    for(int index = 0; index < 5; index++) {
11        while(arr[index] - arr[index]){arr[index]++;}
12        arr[index]*=2;
13        if(arr[3]) {printf("0 element detected!\n");}
14        index = index + T;
15    }
16    return 0;
17 }

```

3 marks

A. Line 9 B. Line 13 C. Line 11 D. Line 5 E. Line 14

Q.2) Which of the following statements are **true** for the given code A ?

Note: Assume that all operators below follow the left-to-right associativity.

Solution: A, C, D

Code A

```

1 #include <stdio.h>
2
3 int X = 5;
4
5 int main() {
6     int Y = X^X;
7     int a, b, c;
8     scanf("%d %d %d", &a, &b, &c);
9
10    if((Y*a + b)&Y&&(c||b)) {
11        printf("First branch!\n");
12    } else if(~Y||b) {
13        printf("Second branch!\n");
14    } else if (b+c == a) {
15        printf("Third branch!\n");
16    } else {
17        printf("Last branch!\n");
18    }
19    return 0;
20 }

```

3 marks

A. The expression 'b' in line 13 of code A will not be evaluated, as it will exercise an optimization where it only needs to evaluate the expression before the logical OR, which is always true.

B. The expressions 'Y' and '(c||b)' will not be evaluated in line 11 of code A, since 'Y * a + b' would always evaluate to 0 for any choice of a, b (as it's equivalent to 'Y * (a + b)').

C. Code A will always print "Second branch!".

D. In code A subsequent control statements after line 13 can be eliminated by the compiler.

Q.3) Which of the following statements are **true** for the given code B ?

Note: Assume that all operators below follow the left-to-right associativity.

Solution: B, C

Code B

```
1 #include <stdio.h>
2
3 int quadratic(int x) {
4     return x*x - 2*x + 3;
5 }
6
7 int main() {
8
9     int a;
10    scanf("%d", &a);
11
12    switch(quadratic(a)) {
13        case 0:
14            printf("First branch!\n");
15        case 3:
16            printf("Second branch!\n");
17            break;
18        case 19:
19            printf("Third branch!\n");
20            break;
21        case 11:
22            printf("Fourth branch!\n");
23            break;
24        default:
25            printf("Last branch!\n");
26    }
27    return 0;
28 }
```

3 marks

- A. In code B, statements corresponding to cases which will never be executed at run time for any input cannot be removed from code at compile time.
- B. Cases 0 and 19 will never occur.
- C. Cases 3 and 11 may occur for certain inputs.
- D. Line 15 in code B will get executed only if line 13 in the same code gets executed, since there is no 'break' statement between lines 14 and 15. Since line 13 can never get executed for any integral input to the 'quadratic()' function, it follows that line 15 will also never get executed.